Parsing expression grammar

In computer science, a **parsing expression grammar** (**PEG**) is a type of analytic formal grammar, i.e. it describes a formal language in terms of a set of rules for recognizing strings in the language. The formalism was introduced by Bryan Ford in 2004[1] and is closely related to the family of top-down parsing languages introduced in the early 1970s. Syntactically, PEGs also look similar to context-free grammars (CFGs), but they have a different interpretation: the choice operator selects the first match in PEG, while it is ambiguous in CFG. This is closer to how string recognition tends to be done in practice, e.g. by a recursive descent parser.

Unlike CFGs, PEGs cannot be ambiguous; a string has exactly one valid parse tree or none. It is conjectured that there exist context-free languages that cannot be recognized by a PEG, but this is not yet proven.[1] PEGs are well-suited to parsing computer languages (and artificial human languages such as Lojban) where multiple interpretation alternatives can be disambiguated locally, but are less likely to be useful for parsing natural languages where disambiguation may have to be global.[2]

Definition

A **parsing expression** is a kind of pattern that each string may either **match** or **not match**. In case of a match, there is a unique prefix of the string (which may be the whole string, the empty string, or something in between) which has been *consumed* by the parsing expression; this prefix is what one would usually think of as having matched the expression. However, whether a string matches a parsing expression *may* (because of look-ahead predicates) depend on parts of it which come after the consumed part. A **parsing expression language** is a set of all strings that match some specific parsing expression.[1]:Sec.3.4

A **parsing expression grammar** is a collection of named parsing expressions, which may reference each other. The effect of one such reference in a parsing expression is as if the whole referenced parsing expression was given in place of the reference. A parsing expression grammar also has a designated **starting expression**; a string matches the grammar if it matches its starting expression.

An element of a string matched is called a *terminal symbol*, or **terminal** for short. Likewise the names assigned to parsing expressions are called *nonterminal symbols*, or **nonterminals** for short. These terms would be descriptive for generative grammars, but in the case of parsing expression grammars they are merely terminology, kept mostly because of being near ubiquitous in discussions of parsing algorithms.

Syntax

Both *abstract* and *concrete* syntaxes of parsing expressions are seen in the literature, and in this article. The abstract syntax is essentially a mathematical formula and primarily used in theoretical contexts, whereas concrete syntax parsing expressions could be used directly to control a parser. The primary concrete syntax is that defined by Ford,[1]:Fig.1 although many tools have their own dialect of this. Other tools[3] can be closer to using a programming-language native encoding of abstract syntax parsing expressions as their concrete syntax.

Atomic parsing expressions

The two main kinds of parsing expressions not containing another parsing expression are individual terminal symbols and nonterminal symbols. In concrete syntax, terminals are placed inside quotes (single or double), whereas identifiers not in quotes denote nonterminals:

```
"terminal" Nonterminal 'another terminal'
```

In the abstract syntax there is no formalised distinction, instead each symbol is supposedly defined as either terminal or nonterminal, but a common convention is to use upper case for nonterminals and lower case for terminals.

The concrete syntax also has a number of forms for classes of terminals:

- A `.` (period) is a parsing expression matching any single terminal.
- Brackets around a list of characters `[abcde]` form a parsing expression matching one of the numerated characters. As in regular expressions, these classes may also include ranges `[0-9A-Za-z]` written as a hyphen with the range endpoints before and after it. (Unlike the case in regular expressions, bracket character classes do not have `^` for negation; that end can instead be had via not-predicates.)
- Some dialects have further notation for predefined classes of characters, such as letters, digits, punctuation marks, or spaces; this is again similar to the situation in regular expressions.

In abstract syntax, such forms are usually formalised as nonterminals whose exact definition is elided for brevity; in Unicode, there are tens of thousands of characters that are letters. Conversely, theoretical discussions sometimes introduce atomic abstract syntax for concepts that can alternatively be expressed using composite parsing expressions. Examples of this include:

- the empty string ε (as a parsing expression, it matches every string and consumes no characters),
- end of input *E* (concrete syntax equivalent is `!.`), and
- failure ⊥ (matches nothing).

In the concrete syntax, quoted and bracketed terminals have backslash escapes, so that "line feed or carriage return" may be written `[\n\r]`. The abstract syntax counterpart of a quoted terminal of length greater than one would be the sequence of those terminals; `"bar"` is the same as `"b" "a" "r"`. The primary concrete syntax assigns no distinct meaning to terminals depending on whether they use single or double quotes, but some dialects treat one as case-sensitive and the other as case-insensitive.

Composite parsing expressions

Given any existing parsing expressions e, e_1, and e_2, a new parsing expression can be constructed using the following operators:

- *Sequence*: e_1 e_2
- *Ordered choice*: e_1 / e_2
- *Zero-or-more*: e*
- *One-or-more*: e+
- *Optional*: e?
- *And-predicate*: &e
- *Not-predicate*: !e
- *Group*: (e)

Operator priorities are as follows, based on Table 1 in:[1]

Operator	Priority
(e)	5
e*, e+, e?	4
&e, !e	3
e_1 e_2	2
e_1 / e_2	1

Grammars

In the concrete syntax, a parsing expression grammar is simply a sequence of nonterminal definitions, each of which has the form

```
Identifier LEFTARROW Expression
```

The `Identifier` is the nonterminal being defined, and the `Expression` is the parsing expression it is defined as referencing. The `LEFTARROW` varies a bit between dialects, but is generally some left-pointing arrow or assignment symbol, such as `<-`, ←, `:=`, or `=`. One way to understand it is precisely as making an assignment or definition of the nonterminal. Another way to understand it is as a contrast to the right-pointing arrow → used in the rules of a context-free grammar; with parsing expressions the flow of information goes from expression to nonterminal, not nonterminal to expression.

As a mathematical object, a parsing expression grammar is a tuple (N, Σ, P, e_S), where N is the set of nonterminal symbols, Σ is the set of terminal symbols, P is a function from N to the set of parsing expressions on $N \cup \Sigma$, and e_S is the starting parsing expression. Some concrete syntax dialects give the starting expression explicitly,[4] but the primary concrete syntax instead has the implicit rule that the first nonterminal defined is the starting expression.

It is worth noticing that the primary dialect of concrete syntax parsing expression grammars does not have an explicit definition terminator or separator between definitions, although it is customary to begin a new definition on a new line; the `LEFTARROW` of the next definition is sufficient for finding the boundary, if one adds the constraint that a nonterminal in an `Expression` must not be followed by a `LEFTARROW`. However, some dialects may allow an explicit terminator, or outright require[4] it.

Example

This is a PEG that recognizes mathematical formulas that apply the basic five operations to non-negative integers.

```
Expr    ← Sum
Sum     ← Product (('+' / '-') Product)*
Product ← Power (('*' / '/') Power)*
Power   ← Value ('^' Power)?
Value   ← [0-9]+ / '(' Expr ')'
```

In the above example, the terminal symbols are characters of text, represented by characters in single quotes, such as `'('` and `')'`. The range `[0–9]` is a shortcut for the ten characters from `'0'` to `'9'`. (This range syntax is the same as the syntax used by regular expressions.) The nonterminal symbols are the ones that expand to other rules: *Value*, *Power*, *Product*, *Sum*, and *Expr*. Note that rules *Sum* and *Product* don't lead to desired left-associativity of these operations (they don't deal with associativity at all, and it has to be handled in post-processing step after parsing), and the *Power* rule (by referring to itself on the right) results in desired right-associativity of exponent. Also note that a rule like `Sum ← Sum (('+' / '-') Product)?` (with intention to achieve left-associativity) would cause infinite recursion, so it cannot be used in practice even though it can be expressed in the grammar.

Semantics

The fundamental difference between context-free grammars and parsing expression grammars is that the PEG's choice operator is *ordered*. If the first alternative succeeds, the second alternative is ignored. Thus ordered choice is not commutative, unlike unordered choice as in context-free grammars. Ordered choice is analogous to soft cut operators available in some logic programming languages.

The consequence is that if a CFG is transliterated directly to a PEG, any ambiguity in the former is resolved by deterministically picking one parse tree from the possible parses. By carefully choosing the order in which the grammar alternatives are specified, a programmer has a great deal of control over which parse tree is selected.

Parsing expression grammars also add the and- and not- syntactic predicates. Because they can use an arbitrarily complex sub-expression to "look ahead" into the input string without actually consuming it, they provide a powerful syntactic lookahead and disambiguation facility, in particular when reordering the alternatives cannot specify the exact parse tree desired.

Operational interpretation of parsing expressions

Each nonterminal in a parsing expression grammar essentially represents a parsing function in a recursive descent parser, and the corresponding parsing expression represents the "code" comprising the function. Each parsing function conceptually takes an input string as its argument, and yields one of the following results:

- *success*, in which the function may optionally move forward or *consume* one or more characters of the input string supplied to it, or
- *failure*, in which case no input is consumed.

An atomic parsing expression consisting of a single **terminal** (i.e. literal) succeeds if the first character of the input string matches that terminal, and in that case consumes the input character; otherwise the expression yields a failure result. An atomic parsing expression consisting of the **empty** string always trivially succeeds without consuming any input.

An atomic parsing expression consisting of a **nonterminal** *A* represents a recursive call to the nonterminal-function *A*. A nonterminal may succeed without actually consuming any input, and this is considered an outcome distinct from failure.

The **sequence** operator $e_1\ e_2$ first invokes e_1, and if e_1 succeeds, subsequently invokes e_2 on the remainder of the input string left unconsumed by e_1, and returns the result. If either e_1 or e_2 fails, then the sequence expression $e_1\ e_2$ fails (consuming no input).

The **choice** operator $e_1\ /\ e_2$ first invokes e_1, and if e_1 succeeds, returns its result immediately. Otherwise, if e_1 fails, then the choice operator backtracks to the original input position at which it invoked e_1, but then calls e_2 instead, returning e_2's result.

The **zero-or-more**, **one-or-more**, and **optional** operators consume zero or more, one or more, or zero or one consecutive repetitions of their sub-expression *e*, respectively. Unlike in context-free grammars and regular expressions, however, these operators *always* behave greedily, consuming as much input as possible and never backtracking. (Regular expression matchers may start by matching greedily, but will then backtrack and try shorter matches if they fail to match.) For example, the expression a* will always consume as many a's as are consecutively available in the input string, and the expression (a* a) will always fail because the first part (a*) will never leave any a's for the second part to match.

The **and-predicate** expression &*e* invokes the sub-expression *e*, and then succeeds if *e* succeeds and fails if *e* fails, but in either case *never consumes any input*.

The **not-predicate** expression !*e* succeeds if *e* fails and fails if *e* succeeds, again consuming no input in either case.

More examples

The following recursive rule matches standard C-style if/then/else statements in such a way that the optional "else" clause always binds to the innermost "if", because of the implicit prioritization of the '/' operator. (In a context-free grammar, this construct yields the classic dangling else ambiguity.)

```
S ← 'if' C 'then' S 'else' S / 'if' C 'then' S
```

The following recursive rule matches Pascal-style nested comment syntax, `(* which can (* nest *) like this *)`. Recall that `.` matches any single character.

```
C     ← Begin N* End
Begin ← '(*'
End   ← '*)'
N     ← C / (!Begin !End .)
```

The parsing expression `foo &(bar)` matches and consumes the text "foo" but only if it is followed by the text "bar". The parsing expression `foo !(bar)` matches the text "foo" but only if it is *not* followed by the text "bar". The expression `!(a+ b) a` matches a single "a" but only if it is not part of an arbitrarily long sequence of a's followed by a b.

The parsing expression `('a'/'b')*` matches and consumes an arbitrary-length sequence of a's and b's. The production rule `S ← 'a' ''S''? 'b'` describes the simple context-free "matching language" $\{a^n b^n : n \geq 1\}$. The following parsing expression grammar describes the classic non-context-free language $\{a^n b^n c^n : n \geq 1\}$:

```
S ← &(A 'c') 'a'+ B !.
A ← 'a' A? 'b'
B ← 'b' B? 'c'
```

Implementing parsers from parsing expression grammars

Any parsing expression grammar can be converted directly into a recursive descent parser.[5] Due to the unlimited lookahead capability that the grammar formalism provides, however, the resulting parser could exhibit exponential time performance in the worst case.

It is possible to obtain better performance for any parsing expression grammar by converting its recursive descent parser into a *packrat parser*, which always runs in linear time, at the cost of substantially greater storage space requirements. A packrat parser[5] is a form of parser similar to a recursive descent parser in construction, except that during the parsing process it memoizes the intermediate results of all invocations of the mutually recursive parsing functions, ensuring that each parsing function is only invoked at most once at a given input position. Because of this memoization, a packrat parser has the ability to parse many context-free grammars and *any* parsing expression grammar (including some that do not represent context-free languages) in linear time. Examples of memoized recursive descent parsers are known from at least as early as 1993.[6] This analysis of the performance of a packrat parser assumes that enough memory is available to hold all of the memoized results; in practice, if there is not enough memory, some parsing functions might have to be invoked more than once at the same input position, and consequently the parser could take more than linear time.

It is also possible to build LL parsers and LR parsers from parsing expression grammars, with better worst-case performance than a recursive descent parser without memoization, but the unlimited lookahead capability of the grammar formalism is then lost. Therefore, not all languages that can be expressed using parsing expression grammars can be parsed by LL or LR parsers.

Bottom-up PEG parsing

A pika parser[7] uses dynamic programming to apply PEG rules bottom-up and right to left, which is the inverse of the normal recursive descent order of top-down, left to right. Parsing in reverse order solves the left recursion problem, allowing left-recursive rules to be used directly in the grammar without being rewritten into non-left-recursive form, and also confers optimal error recovery capabilities upon the parser, which historically proved difficult to achieve for recursive descent parsers.

Advantages

No compilation required

Many parsing algorithms require a preprocessing step where the grammar is first compiled into an opaque executable form, often some sort of automaton. Parsing expressions can be executed directly (even if it is typically still advisable to transform the human-readable PEGs shown in this article into a more native format, such as S-expressions, before evaluating them).

Compared to regular expressions

Compared to pure regular expressions (i.e., describing a language recognisable using a finite automaton), PEGs are vastly more powerful. In particular they can handle unbounded recursion, and so match parentheses down to an arbitrary nesting depth; regular expressions can at best keep track of nesting down to some fixed depth, because a finite automaton (having a finite set of internal states) can only distinguish finitely many different nesting depths. In more theoretical terms, $\{a^n b^n\}_{n \geqslant 0}$ (the language of all strings of zero or more a's, followed by an *equal number* of bs) is not a regular language, but it is easily seen to be a parsing expression language, matched by the grammar

```
start ← AB !.
AB ← ('a' AB 'b')?
```

Here `AB !.` is the starting expression. The `!.` part enforces that the input ends after the `AB`, by saying "there is no next character"; unlike regular expressions, which have magic constraints `$` or `\Z` for this, parsing expressions can express the end of input using only the basic primitives.

The `*`, `+`, and `?` of parsing expressions are similar to those in regular expressions, but a difference is that these operate strictly in a greedy mode. This is ultimately due to `/` being an ordered choice. A consequence is that something can match as a regular expression which does

not match as parsing expression:

```
[ab]?[bc][cd]
```

is both a valid regular expression and a valid parsing expression. As regular expression, it matches `bc`, but as parsing expression it does not match, because the `[ab]?` will match the `b`, then `[bc]` will match the `c`, leaving nothing for the `[cd]`, so at that point matching the sequence fails. "Trying again" with having `[ab]?` match the empty string is explicitly against the semantics of parsing expressions; this is not an edge case of a particular matching algorithm, instead it is the sought behaviour.

Even regular expressions that depend on nondeterminism *can* be compiled into a parsing expression grammar, by having a separate nonterminal for every state of the corresponding DFA and encoding its transition function into the definitions of these nonterminals —

```
A ← 'x' B / 'y' C
```

is effectively saying "from state A transition to state B if the next character is x, but to state C if the next character is y" — but this works because nondeterminism can be eliminated within the realm of regular languages. It would not make use of the parsing expression variants of the repetition operations.

Compared to context-free grammars

PEGs can comfortably be given in terms of characters, whereas context-free grammars (CFGs) are usually given in terms of tokens, thus requiring an extra step of tokenisation in front of parsing proper.[8] An advantage of not having a separate tokeniser is that different parts of the language (for example embedded mini-languages) can easily have different tokenisation rules.

In the strict formal sense, PEGs are likely incomparable to CFGs, but practically there are many things that PEGs can do which pure CFGs cannot, whereas it is difficult to come up with examples of the contrary. In particular PEGs can be crafted to natively resolve ambiguities, such as the "dangling else" problem in C, C++, and Java, whereas CFG-based parsing often needs a rule outside of the grammar to resolve them. Moreover any PEG can be parsed in linear time by using a packrat parser, as described above, whereas parsing according to a general CFG is asymptotically equivalent[9] to boolean matrix multiplication (thus likely between quadratic and cubic time).

One classical example of a formal language which is provably not context-free is the language $\{a^n b^n c^n\}_{n \geqslant 0}$: an arbitrary number of as are followed by an equal number of bs, which in turn are followed by an equal number of cs. This, too, is a parsing expression language, matched by the grammar

```
start ← AB 'c'*
AB    ← 'a' AB 'b' / &(BC !.)
BC    ← ('b' BC 'c')?
```

For AB to match, the first stretch of as must be followed by an equal number of bs, and in addition BC has to match where the as switch to bs, which means those bs are followed by an equal number of cs.

Disadvantages

Memory consumption

PEG parsing is typically carried out via *packrat parsing*, which uses memoization[10][11] to eliminate redundant parsing steps. Packrat parsing requires internal storage proportional to the total input size, rather than to the depth of the parse tree as with LR parsers. Whether this is a significant difference depends on circumstances; if parsing is a service provided as a function then the parser will have stored the full parse tree up until returning it, and already that parse tree will typically be of size proportional to the total input size. If parsing is instead provided as a generator then one might get away with only keeping parts of the parse tree in memory, but the feasibility of this depends on the grammar. A parsing expression grammar can be designed so that only after consuming the full input will the parser discover that it needs to backtrack to the beginning,[12] which again could require storage proportional to total input size.

For recursive grammars and some inputs, the depth of the parse tree can be proportional to the input size,[13] so both an LR parser and a packrat parser will appear to have the same worst-case asymptotic performance. However in many domains, for example hand-written source code, the expression nesting depth has an effectively constant bound quite independent of the length of the program, because expressions nested beyond a certain depth tend to get refactored. When it is not necessary to keep the full parse tree, a more accurate analysis would take the depth of the parse tree into account separately from the input size.[14]

Computational model

In order to attain linear overall complexity, the storage used for memoization must furthermore provide amortized constant time access to individual data items memoized. In practice that is no problem — for example a dynamically sized hash table attains this – but that makes use of pointer arithmetic, so it presumes having a random-access machine. Theoretical discussions of data structures and algorithms have an unspoken tendency to presume a more restricted model (possibly that of lambda calculus, possibly that of Scheme), where a sparse table rather has to be built using trees, and data item access is not constant time. Traditional parsing algorithms such as the LL parser are not affected by this, but it becomes a penalty for the reputation of packrat parsers: they rely on operations of seemingly ill repute.

Viewed the other way around, this says packrat parsers tap into computational power readily available in real life systems, that older parsing algorithms do not understand to employ.

Indirect left recursion

A PEG is called *well-formed*[1] if it contains no *left-recursive* rules, i.e., rules that allow a nonterminal to expand to an expression in which the same nonterminal occurs as the leftmost symbol. For a left-to-right top-down parser, such rules cause infinite regress: parsing will continually expand the same nonterminal without moving forward in the string. Therefore, to allow packrat parsing, left recursion must be eliminated.

Practical significance

Direct recursion, be that left or right, is important in context-free grammars, because there recursion is the only way to describe repetition:

```
Sum   → Term | Sum '+' Term | Sum '−' Term
Args  → Arg | Arg ',' Args
```

People trained in using context-free grammars often come to PEGs expecting to use the same idioms, but parsing expressions can do repetition without recursion:

```
Sum   ← Term ( '+' Term / '−' Term )*
Args  ← Arg ( ',' Arg )*
```

A difference lies in the abstract syntax trees generated: with recursion each `Sum` or `Args` can have at most two children, but with repetition there can be arbitrarily many. If later stages of processing require that such lists of children are recast as trees with bounded degree, for example microprocessor instructions for addition typically only allow two operands, then properties such as left-associativity would be imposed after the PEG-directed parsing stage.

Therefore left-recursion is practically less likely to trouble a PEG packrat parser than, say, an LL(k) context-free parser, unless one insists on using context-free idioms. However, not all cases of recursion are about repetition.

Non-repetition left-recursion

For example, in the arithmetic grammar above, it could seem tempting to express operator precedence as a matter of ordered choice — `Sum / Product / Value` would mean first try viewing as `Sum` (since we parse top–down), second try viewing as `Product`, and only third try viewing as `Value` — rather than via nesting of definitions. This (non-well-formed) grammar seeks to keep precedence order only in one line:

```
Value   ← [0-9.]+ / '(' Expr ')'
Product ← Expr (('*' / '/') Expr)+
Sum     ← Expr (('+' / '−') Expr)+
Expr    ← Sum / Product / Value
```

Unfortunately matching an `Expr` requires testing if a `Sum` matches, while matching a `Sum` requires testing if an `Expr` matches. Because the term appears in the leftmost position, these rules make up a circular definition that cannot be resolved. (Circular definitions that can be

resolved exist—such as in the original formulation from the first example—but such definitions are required not to exhibit pathological recursion.) However, left-recursive rules can always be rewritten to eliminate left-recursion.[2][15] For example, the following left-recursive CFG rule:

```
string-of-a ← string-of-a 'a' | 'a'
```

can be rewritten in a PEG using the plus operator:

```
string-of-a ← 'a'+
```

The process of rewriting *indirectly* left-recursive rules is complex in some packrat parsers, especially when semantic actions are involved.

With some modification, traditional packrat parsing can support direct left recursion,[5][16][17] but doing so results in a loss of the linear-time parsing property[16] which is generally the justification for using PEGs and packrat parsing in the first place. Only the OMeta parsing algorithm[16] supports full direct and indirect left recursion without additional attendant complexity (but again, at a loss of the linear time complexity), whereas all GLR parsers support left recursion.

Unexpected behaviour

A common first impression of PEGs is that they look like CFGs with certain convenience features — repetition operators `*+?` as in regular expressions and lookahead predicates `&!` — plus ordered choice for disambiguation. This understanding can be sufficient when one's goal is to create a parser for a language, but it is not sufficient for more theoretical discussions of the computational power of parsing expressions. In particular the nondeterminism inherent in the unordered choice `|` of context-free grammars makes it very different from the deterministic ordered choice `/`.

The midpoint problem

PEG packrat parsers cannot recognize some unambiguous nondeterministic CFG rules, such as the following:[2]

```
S ← 'x' S 'x' | 'x'
```

Neither LL(k) nor LR(k) parsing algorithms are capable of recognizing this example. However, this grammar can be used by a general CFG parser like the CYK algorithm. However, the *language* in question can be recognised by all these types of parser, since it is in fact a regular language (that of strings of an odd number of x's).

It is instructive to work out exactly what a PEG parser does when attempting to match

```
S ← 'x' S 'x' / 'x'
```

against the string `xxxxxq`. As expected, it recursively tries to match the nonterminal `S` at increasing positions in this string, until failing the match against the `q`, and after that begins to backtrack. This goes as follows:

```
Position:  123456
  String:  xxxxxq
 Results:       ↑ Pos.6: Neither branch of S matches
               ↑ Pos.5: First branch of S fails, second branch succeeds, yielding match of length 1.
              ↑ Pos.4: First branch of S fails, second branch succeeds, yielding match of length 1.
             ↑ Pos.3: First branch of S succeeds, yielding match of length 3.
            ↑ Pos.2: First branch of S fails, because after the S match at 3 comes a q.
              Second branch succeeds, yielding match of length 1.
           ↑ Pos.1: First branch of S succeeds, yielding match of length 3.
```

Matching against a parsing expression is greedy, in the sense that the first success encountered is the only one considered. Even if locally the choices are ordered longest first, there is no guarantee that this greedy matching will find the globally longest match.

Ambiguity detection and influence of rule order on language that is matched

LL(k) and LR(k) parser generators will fail to complete when the input grammar is ambiguous. This is a feature in the common case that the grammar is intended to be unambiguous but is defective. A PEG parser generator will resolve unintended ambiguities earliest-match-first, which may be arbitrary and lead to surprising parses.

The ordering of productions in a PEG grammar affects not only the resolution of ambiguity, but also the *language matched*. For example, consider the first PEG example in Ford's paper[1] (example rewritten in pegjs.org/online notation, and labelled G_1 and G_2):

- G_1: `A = "a" "b" / "a"`
- G_2: `A = "a" / "a" "b"`

Ford notes that *The second alternative in the latter PEG rule will never succeed because the first choice is always taken if the input string ... begins with 'a'.*.[1] Specifically, $L(G_1)$ (i.e., the language matched by G_1) includes the input "ab", but $L(G_2)$ does not. Thus, adding a new option to a PEG grammar can *remove* strings from the language matched, e.g. G_2 is the addition of a rule to the single-production grammar `A = "a" "b"`, which contains a string not matched by G_2. Furthermore, constructing a grammar to match $L(G_1) \cup L(G_2)$ from PEG grammars G_1 and G_2 is not always a trivial task. This is in stark contrast to CFG's, in which the addition of a new production cannot remove strings (though, it can introduce problems in the form of ambiguity), and a (potentially ambiguous) grammar for $L(G_1) \cup L(G_2)$ can be constructed

```
S → start(G1) | start(G2)
```

Theory of parsing expression grammars

It is an open problem to give a concrete example of a context-free language which cannot be recognized by a parsing expression grammar.[1] In particular, it is open whether a parsing expression grammar can recognize the language of palindromes.[18]

The class of parsing expression languages is closed under set intersection and complement, thus also under set union.[1]:Sec.3.4

Undecidability of emptiness

In stark contrast to the case for context-free grammars, it is not possible to generate elements of a parsing expression language from its grammar. Indeed, it is algorithmically undecidable whether the language recognised by a parsing expression grammar is empty! One reason for this is that any instance of the Post correspondence problem reduces to an instance of the problem of deciding whether a parsing expression language is empty.

Recall that an instance of the Post correspondence problem consists of a list $(\alpha_1, \beta_1), (\alpha_2, \beta_2), \ldots, (\alpha_n, \beta_n)$ of pairs of strings (of terminal symbols). The problem is to determine whether there exists a sequence $\{k_i\}_{i=1}^{m}$ of indices in the range $\{1, \ldots, n\}$ such that $\alpha_{k_1}\alpha_{k_2}\cdots\alpha_{k_m} = \beta_{k_1}\beta_{k_2}\cdots\beta_{k_m}$. To reduce this to a parsing expression grammar, let $\gamma_0, \gamma_1, \ldots, \gamma_n$ be arbitrary pairwise distinct equally long strings of terminal symbols (already with 2 distinct symbols in the terminal symbol alphabet, length $\lceil \log_2(n+1) \rceil$ suffices) and consider the parsing expression grammar

$$S \leftarrow \&(A\,!.)\&(B\,!.)(\gamma_1 / \cdots / \gamma_n)^+ \gamma_0$$
$$A \leftarrow \gamma_0 / \gamma_1 A \alpha_1 / \cdots / \gamma_n A \alpha_n$$
$$B \leftarrow \gamma_0 / \gamma_1 B \beta_1 / \cdots / \gamma_n B \beta_n$$

Any string matched by the nonterminal A has the form $\gamma_{k_m}\cdots\gamma_{k_2}\gamma_{k_1}\gamma_0\alpha_{k_1}\alpha_{k_2}\cdots\alpha_{k_m}$ for some indices $k_1, k_2, \ldots, k_m$. Likewise any string matched by the nonterminal B has the form $\gamma_{k_m}\cdots\gamma_{k_2}\gamma_{k_1}\gamma_0\beta_{k_1}\beta_{k_2}\cdots\beta_{k_m}$. Thus any string matched by S will have the form $\gamma_{k_m}\cdots\gamma_{k_2}\gamma_{k_1}\gamma_0\rho$ where $\rho = \alpha_{k_1}\alpha_{k_2}\cdots\alpha_{k_m} = \beta_{k_1}\beta_{k_2}\cdots\beta_{k_m}$.

Practical use

- Python reference implementation CPython introduced a PEG parser in version 3.9 as an alternative to the LL(1) parser and uses just PEG from version 3.10.[19]
- The jq programming language uses a formalism closely related to PEG.
- The Lua authors created LPeg (https://www.inf.puc-rio.br/~roberto/lpeg/), a pattern-matching library that uses PEG instead of regular expressions,[20] as well as the re (https://www.inf.puc-rio.br/~roberto/lpeg/re.html) module which implements a regular-expression-like syntax utilizing the LPeg library.[21]

See also

- Boolean context-free grammar
- Compiler Description Language (CDL)
- Formal grammar
- Regular expression
- Top-down parsing language
- Comparison of parser generators
- Parser combinator

References

1. Ford, Bryan (January 2004). "Parsing Expression Grammars: A Recognition Based Syntactic Foundation" (https://bford.info/pub/lang/peg.pdf) (PDF). *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM. pp. 111–122. doi:10.1145/964001.964011 (https://doi.org/10.1145%2F964001.964011). ISBN 1-58113-729-X.
2. Ford, Bryan (September 2002). "Packrat parsing: simple, powerful, lazy, linear time, functional pearl" (http://pdos.csail.mit.edu/~baford/packrat/icfp02/packrat-icfp02.pdf) (PDF). *ACM SIGPLAN Notices*. **37** (9). doi:10.1145/583852.581483 (https://doi.org/10.1145%2F583852.581483).
3. Sirthias, Mathias. "Parboiled: Rule Construction in Java" (https://github.com/sirthias/parboiled/wiki/Rule-Construction-in-Java). *GitHub*. Retrieved 13 January 2024.
4. Kupries, Andreas. "pt::peg_language - PEG Language Tutorial" (https://core.tcl-lang.org/tcllib/doc/tcllib-1-21/embedded/md/tcllib/files/modules/pt/pt_peg_language.md). *Tcl Library Source Code*. Retrieved 14 January 2024.
5. Ford, Bryan (September 2002). *Packrat Parsing: a Practical Linear-Time Algorithm with Backtracking* (http://pdos.csail.mit.edu/~baford/packrat/thesis) (Thesis). Massachusetts Institute of Technology. Retrieved 2007-07-27.
6. Merritt, Doug (November 1993). "Transparent Recursive Descent" (http://compilers.iecc.com/comparch/article/93-11-012). Usenet group comp.compilers. Retrieved 2009-09-04.
7. Hutchison, Luke A. D. (2020). "Pika parsing: parsing in reverse solves the left recursion and error recovery problems". arXiv:2005.06444 (https://arxiv.org/abs/2005.06444) [cs.PL (https://arxiv.org/archive/cs.PL)].
8. CFGs *can* be used to describe the syntax of common programming languages down to the character level, but doing so is rather cumbersome, because the standard tokenisation rule that a token consists of the longest consecutive sequence of characters of the same kind does not mesh well with the nondeterministic side of CFGs. To formalise that whitespace between two adjacent tokens is mandatory if the characters on both sides of the token boundary are letters, but optional if they are non-letters, a CFG needs multiple variants of most nonterminals, to keep track of what kind of character has to be at the boundary. If there are 3 different kinds of non-whitespace characters, that adds up to $3^2 = 9$ possible variants per nonterminal — significantly bloating the grammar.
9. Lee, Lillian (January 2002). "Fast Context-free Grammar Parsing Requires Fast Boolean Matrix Multiplication". *J. ACM*. **49** (1): 1–15. arXiv:cs/0112018 (https://arxiv.org/abs/cs/0112018). doi:10.1145/505241.505242 (https://doi.org/10.1145%2F505241.505242).
10. Ford, Bryan. "The Packrat Parsing and Parsing Expression Grammars Page" (https://bford.info/packrat/). *BFord.info*. Retrieved 23 Nov 2010.
11. Jelliffe, Rick (10 March 2010). "What is a Packrat Parser? What are Brzozowski Derivatives?" (https://web.archive.org/web/20110728124552/http://broadcast.oreilly.com/2010/03/what-is-a-packrat-parser-what.html). Archived from the original (http://broadcast.oreilly.com/2010/03/what-is-a-packrat-parser-what.html) on 28 July 2011

com/2010/03/what-is-a-packrat-parser-what.html) on 28 July 2011.
12. For example, there could at the very end of input be a directive to the effect that “in this file, comma is a decimal separator, so all those function calls f(3,14*r) you thought had two arguments? They don't. Now go back to the start of input and parse it all again.” Arguably that would be a poor design of the input language, but the point is that parsing expression grammars are powerful enough to handle this, purely as a matter of syntax.
13. for example, the LISP expression (x (x (x (x ....))))
14. This is similar to a situation which arises in graph algorithms: the Bellman–Ford algorithm and Floyd–Warshall algorithm appear to have the same running time ($O(|V|^3)$) if only the number of vertices is considered. However, a more precise analysis which accounts for the number of edges as a separate parameter assigns the Bellman–Ford algorithm a time of $O(|V| * |E|)$, which is quadratic for sparse graphs with $|E| \in O(|V|)$.
15. Aho, A.V.; Sethi, R.; Ullman, J.D. (1986). *Compilers: Principles, Techniques, and Tools* (https://archive.org/details/compilersprincip0000ahoa/). Boston, MA, USA: Addison-Wesley Longman. ISBN 0-201-10088-6.
16. Warth, Alessandro; Douglass, James R.; Millstein, Todd (January 2008). "Packrat Parsers Can Support Left Recursion" (http://www.vpri.org/pdf/tr2007002_packrat.pdf) (PDF). *Proceedings of the 2008 ACM SIGPLAN symposium on Partial evaluation and semantics-based program manipulation*. PEPM '08. ACM. pp. 103–110. doi:10.1145/1328408.1328424 (https://doi.org/10.1145%2F1328408.1328424). ISBN 9781595939777. Retrieved 2008-10-02.
17. Steinmann, Ruedi (March 2009). "Handling Left Recursion in Packrat Parsers" (https://web.archive.org/web/20110706232049/http://n.ethz.ch/~ruediste/packrat.pdf) (PDF). *n.ethz.ch*. Archived from the original (http://n.ethz.ch/~ruediste/packrat.pdf) (PDF) on 2011-07-06.
18. Loff, Bruno; Moreira, Nelma; Reis, Rogério (2020-02-14). "The computational power of parsing expression grammars". arXiv:1902.08272 (https://arxiv.org/abs/1902.08272) [cs.FL (https://arxiv.org/archive/cs.FL)].
19. "PEP 617 – New PEG parser for CPython | peps.python.org" (https://peps.python.org/pep-0617/). *peps.python.org*. Retrieved 2023-01-16.
20. Ierusalimschy, Roberto (10 March 2009). "A text pattern-matching tool based on Parsing Expression Grammars" (https://onlinelibrary.wiley.com/doi/10.1002/spe.892). *Software: Practice and Experience*. **39** (3): 221–258. doi:10.1002/spe.892 (https://doi.org/10.1002%2Fspe.892). ISSN 0038-0644 (https://search.worldcat.org/issn/0038-0644).
21. Ierusalimschy, Roberto. "LPeg.re - Regex syntax for LPEG" (https://www.inf.puc-rio.br/~roberto/lpeg/re.html). *www.inf.puc-rio.br*.

External links

- Converting a string expression into a lambda expression using an expression parser (https://web.archive.org/web/20131103083443/http://mathosproject.com/updates/convert-a-string-expression-into-a-lambda-expression/)
- The Packrat Parsing and Parsing Expression Grammars Page (http://bford.info/packrat/)
- The constructed language Lojban has a fairly large PEG grammar (http://www.digitalkingdom.org/~rlpowell/hobbies/lojban/grammar/) allowing unambiguous parsing of Lojban text.
- An illustrative implementation of a PEG in Guile scheme (https://www.gnu.org/software/guile/manual/html_node/PEG-Parsing.html)